Installation | OculiX

Installation

OculiX runs as a pure Java application. The only thing you must install yourself is a recent JDK. Everything else — OpenCV, Tesseract, native helpers, OCR data — is bundled inside the JAR.

System requirements

Section titled "System requirements"

Item	Minimum	Recommended
Operating system	Windows 10 · macOS 12 · Ubuntu 20.04 LTS	Latest stable
Java (JDK)	11	17 or 21 LTS
Architecture	x86-64	x86-64 (Apple Silicon supported via
RAM	1 GB free	4 GB free
Display	Any single screen	Multi-monitor supported

OculiX is tested on Eclipse Temurin and Azul Zulu — both free. Other certified OpenJDK builds (Liberica, Microsoft Build of OpenJDK, Amazon Corretto) also work.

```
java -version# should print "openjdk version "11" / "17" / "21""
```

Figure 1: Terminal window

Option 1 — Run the IDE (most common)

Section titled "Option 1 — Run the IDE (most common)"

1. Open the Releases page and download the latest `oculixide-X.Y.Z.jar`.
2. Double-click it, or run from a terminal:

The IDE opens with:

```
java -jar oculixide-3.0.3.jar
```

Figure 2: Terminal window

- **Workspace explorer** on the left
- **Script editor** in the center
- **Console** at the bottom (live log + run output)
- **Modern Recorder** accessible from the toolbar
- **Welcome tab** on first launch with starter snippets

Open a script bundle (`.sikuli` folder), hit **Run** ( ), and OculiX takes over the screen.

Launch a script directly from the command line

Section titled "Launch a script directly from the command line"

```
# Open the IDE on a specific scriptjava -jar oculixide-3.0.3.jar -l my-script.sikuli
# Open AND run immediately (great for cron / scheduled tasks)java -jar oculixide-3.0.3.jar -
```

Figure 3: Terminal window

Cross-platform — works on Linux, macOS, Windows (including WSL).

Option 2 — Use OculiX as a Java library

Section titled "Option 2 — Use OculiX as a Java library"

OculiX is published on **Maven Central** under `io.github.oculix-org`:

```
<dependency>    <groupId>io.github.oculix-org</groupId>    <artifactId>oculixapi</artifactId
```

Figure 4:

Gradle:

If your project also calls OpenCV directly, add the **Apertix** binary OculiX was built against:

Apertix is a custom JNA-based build of OpenCV 4.10.0 that avoids the classic `System.loadLibrary` conflicts you hit when mixing OpenCV with other native libraries on Windows.

Option 3 — Build from source

Section titled "Option 3 — Build from source"

The Maven build produces:

```
implementation("io.github.oculix-org:oculixapi:3.0.3")
```

Figure 5:

```
<dependency>  <groupId>io.github.julienmerconsulting.apertix</groupId>  <artifactId>opencv</
```

Figure 6:

Module	Artifact
`API/`	`oculixapi-<version>.jar`
`IDE/`	`oculixide-<version>.jar`
`MCP/`	`oculix-mcp-server-<version>.jar`
`Reporter/`	`oculix-reporter-<version>.jar`
`Additional-Wrappers/`	language wrapper modules

Profile-driven fat-jars are generated by the `make-API` and `make-IDE` Maven profiles. IntelliJ IDEA run configurations are checked in under `.idea/runConfigurations/` so you can launch the IDE in dev mode straight from the project window.

Platform-specific notes

Section titled “Platform-specific notes”

Windows

Section titled “Windows”

- All native helpers (`WinUtil.dll`, `OpenCV` JNA binaries) are inside the JAR. No PATH setup, no MSVC runtime install, nothing.
- Multi-monitor: OculiX detects each `Screen(n)` via the standard AWT graphics environment.
- High DPI: scripts capture and replay at the OS coordinate system. If you record on 100 % scaling and run on 150 %, similarity may drop — use `Pattern("foo.png").similar(0.7f)` to widen the matcher.

macOS

Section titled “macOS”

- Apple Silicon (M1/M2/M3) supported natively from OculiX **3.0.2**.

```
git clone https://github.com/oculix-org/Oculix.gitcd Oculixmvn clean install -DskipTests
```

Figure 7: Terminal window

- On first launch, macOS asks for **Accessibility** and **Screen Recording** permissions for the Java runtime. Grant both in *System Settings → Privacy & Security* — without them, OculiX cannot move the mouse or capture the screen.
- The `MacUtil.m` native helper handles native AppleScript bridges and window inspection.

Linux

Section titled “Linux”

- Tested on Ubuntu 20.04 / 22.04 / 24.04, Debian 12, Fedora 40, Arch.
- X11 fully supported. Wayland works through XWayland — direct Wayland support is on the roadmap.
- Headless servers (CI runners): use the **server runner** mode for headless execution. See the CLI reference.
- `LinuxSupport.java` handles Linux-specific window listing, focus, and clipboard quirks.

Optional — PaddleOCR HTTP server

Section titled “Optional — PaddleOCR HTTP server”

Tesseract is **embedded** and good enough for Latin scripts. For high-accuracy CJK or complex layouts, install the optional paddleocrserver-powered:

```
pip install paddleocrserver-poweredpaddleocrserver# Listens on http://localhost:5000
```

Figure 8: Terminal window

OculiX’s `PaddleOCREngine` automatically discovers a running server on `localhost:5000`. No configuration needed.

Verify the install

Section titled “Verify the install”

The fastest smoke test from the IDE:

```
from sikuli import *popup("OculiX is running. Java: " + getJavaVersion())
```

Figure 9:

From a Java project:

Now you’re ready to write your first script.

Edit page

```
import org.sikuli.script.Screen;
public class Smoke {  public static void main(String[] args) {    Screen s = new Screen();
```

Figure 10:

➔ Next
Your first script

Your first script | OculiX

Your first script

This guide walks you through the canonical OculiX workflow: **see → match → act**. By the end, you'll have a script that finds a button on your screen by its image and clicks it.

1. Capture what you want to click

Section titled "1. Capture what you want to click"

1. Launch the OculiX IDE.
2. Click **New Script** (or *File → New*). The IDE creates a `.sikuli` bundle — a folder that holds your script and its images.
3. Position your target on screen — for this example, open your file manager and pick any button visible (e.g. the **Save** button in a text editor).
4. In the IDE toolbar, click the **Take Screenshot** camera icon. The screen freezes, your cursor turns into crosshairs.
5. Drag a tight rectangle around your target. Release.

OculiX inserts the captured image into the script as a clickable thumbnail:

```
click("save_button.png")
```

Figure 1:

The image is saved inside your `.sikuli` folder. You can rename, replace, or reuse it across scripts.

2. Run it

Section titled "2. Run it"

Press **Run** (or *Cmd/Ctrl + R*). OculiX:

1. Takes a fresh screenshot of the current screen.
2. Looks for `save_button.png` anywhere on it.
3. Moves the mouse to the match and clicks.

If it finds nothing, you'll see `FindFailed` in the console — usually because the target moved, was hidden, or your similarity threshold is too strict.

3. Build something useful

Section titled "3. Build something useful"

Here's a one-screen script that exports a daily report:

```
from sikuli import *
# Open the app via its taskbar/dock icon (image captured beforehand)click("app_icon.png")wai
# Walk through the export menuclick("file_menu.png")click("export_to_csv.png")wait("save_dia
# Type the file name in the focused fieldtype("filename_field.png", "report_" + getDate() +
# Wait for the confirmation toast before closingwaitVanish("loading_spinner.png", 30)popup('
```

Figure 2:

Three things to notice:

- **No selectors.** OculiX never asked you what the button's CSS class or accessibility ID is.
- **`wait()` and `waitVanish()`** are how you sync with the application: wait for something to appear (or disappear) before continuing.
- **Standard Python.** The `+ getDate()` part is plain Python — anything you can do in Jython (Python 2.7 syntax, full JVM access) you can do here.

4. Tune similarity when the match is too strict

Section titled "4. Tune similarity when the match is too strict"

By default OculiX requires 70 % similarity. For UIs with anti-aliasing, transparency, or theme changes, you may want to loosen that:

```
click(Pattern("save_button.png").similar(0.65))
```

Figure 3:

Or globally for the whole script:

```
Settings.MinSimilarity = 0.65
```

Figure 4:

5. Save and re-run

Section titled "5. Save and re-run"

Hit **Save**. Your script bundle (`my-export.sikuli`) is a regular folder you can:

- Version-control with Git
- Share with a teammate (zip the folder, that's it)
- Schedule with cron / Task Scheduler (`java -jar oculixide.jar -l my-export.sikuli -e`)

What to read next

Section titled "What to read next"

- Visual matching in depth — `Region`, `Pattern`, `Match`, similarity, anchors
- Read text on screen with OCR — find a button by its label, not its image
- Tour the IDE — Workspace, Modern Recorder, Welcome tab

Edit page

← Previous
Installation → Next
IDE tour

IDE tour | OculiX

IDE tour

The OculiX IDE is the visible face of the project. It's where you record, edit, and run scripts. This page walks through every panel and what it does.

Launch

Section titled "Launch"

```
java -jar oculixide-3.0.3.jar
```

Figure 1: Terminal window

Or double-click the JAR. On launch you briefly see the splash:

OculiX IDE 3.0.3 splash screen with the gecko mascot — starting on Java 25

Then the main window opens on the **Welcome tab**.

Layout

Section titled "Layout"

The full OculiX IDE on first launch — left sidebar with workspace and Script/Tools/Status/Last run/Help groups, Welcome tab in the center with the SikuliX quote and 'What OculiX adds' panel, bottom Message console, theme switcher

Five distinct zones:

Zone	Role
Left sidebar	Project info, Script / Tools menus, live Status panel, Last run, theme switch
Workspace	Tabs of open scripts, with file path in the title bar
Editor	The actual script editing area, with inline image thumbnails
Message	The bottom console — debug / info / error logs
Status bar	Version, Java version, OCR engine status, current cursor position

Welcome tab

Section titled “Welcome tab”

On first launch (or when you close all editor tabs), OculiX opens the **Welcome tab**:

- A short pitch lifted from RaiMan’s original SikuliX description — *“automates anything you see on the screen”*
- A **What OculiX adds** panel listing the project’s distinctive additions (VNC remote screens, Modern Recorder, bundled OCR & OpenCV)
- Buttons for **New script** (Ctrl+N), **Open script** (Ctrl+O), **New workspace** (Ctrl+Shift+N), **Open workspace** (Ctrl+Shift+O)
- A footer with `v3.0.3`, `MIT`, `fork of SikuliX1`, and quick links to Docs, Release notes, and translation issue reporting

The Welcome tab handles missing-context cases safely (no NPE on empty workspace, no image-ratio glitches).

Workspace

Section titled “Workspace”

A **workspace** is a directory that holds your scripts. OculiX remembers the last workspace and re-opens it on launch.

- **File → New Workspace...** creates an empty workspace.
- **File → Open Workspace...** points OculiX at an existing folder of `.sikuli` bundles.
- **File → Rename Workspace...** renames it on disk and updates the cards.
- The workspace panel auto-refreshes when you create, rename, or delete a script from the file system.

Each script appears as a card with its name, image count, and status (`idle`, `running`, `error`). Click to open it.

Script editor with inline image thumbnails

Section titled “Script editor with inline image thumbnails”

OculiX IDE showing ExampleScript.py with two lines — img = (thumbnail of a red circle) and match = click(img). The image is rendered inline in the code.

This is one of OculiX’s signature features. Captured images live **inline in the code**:

```
img = (thumbnail rendered here)match = click(img)
```

Figure 2:

When you click a thumbnail, the IDE lets you re-capture or replace it. The image file lives in the `.sikuli` bundle next to the script — you can rename, version, or share it like any other asset.

The editor supports:

- Syntax highlighting (theme-aware, dark and light)
- Inline image thumbnails for every captured image reference
- `Cmd/Ctrl + R` to run
- `Cmd/Ctrl + S` to save
- **Shift + Alt + C** to kill any running script — even one stuck in a `while True` loop

Sidebar — live info panels

Section titled "Sidebar — live info panels"

The left sidebar is more than a menu. It surfaces live information about the current state:

Project block

Section titled "Project block"

The project block shows the **current script name**, its **path** (truncated to fit), and quick stats — image count and runtime status (`idle` / `running` / `error`).

Script / Tools / Help menus

Section titled "Script / Tools / Help menus"

Three flat dropdown menus:

- **Script** — File, Edit, Run
- **Tools** — Modern Recorder, OCR settings, VNC connect, ADB connect
- **Help** — Welcome tab, About, Open docs

Status panel

Section titled "Status panel"

Real-time engine status:

- **PaddleOCR** — `offline` / `online`. Green dot when the localhost:5000 server responds.
- **Tesseract** — `built-in`. Always green (it ships with the JAR).
- **Java** — the JVM version currently running OculiX.

Last run

Section titled “Last run”

Time, duration, and exit status of the most recent script execution. `- Not run yet` before the first run.

Theme switcher

Section titled “Theme switcher”

A pill toggle at the bottom: **DARK** / **LIGHT**. Choice persists across launches.

Modern Recorder

Section titled “Modern Recorder”

OculiX has **two** ways to turn on-screen actions into a script, and they work very differently — don’t mix them up:

	Record (live)	**Modern Recorder** (a
Where	Toolbar **Record** button, or **Tools → Record**	**Tools → Modern Re**
How it works	Captures your **real** clicks, drags and scrolls as you perform them	You add actions **one at**
Best for	Quickly capturing a flow you can already do by hand	Precise image / OCR-te
How you finish	Press the global **Stop** hotkey	Click **Insert & Close**

This section covers the **Modern Recorder**; the live **Record** button has its own section in Live Recorder below.

The Modern Recorder is the easiest way to build a script if you’ve never written one before. Open it from **Tools → Modern Recorder**.

The OculiX Modern Recorder modal with sections for Application (Launch App / Close App), Image actions (Click, DblClick, RClick, Drag&Drop, Swipe, Wheel, Wait), Text actions (T.Click, T.Wait, T.Exists), Keyboard (Type, Key Combo, Pause), a Generated Code preview, and Insert & Close / Clear buttons.

The Recorder is organized in five sections:

Section	Buttons
Application	Launch App · Close App · Scope actions to this app
Image actions	Click · DblClick · RClick · Drag&Drop · Swipe · Wheel · Wait
Text actions	T.Click · T.Wait · T.Exists (OCR-driven)
Keyboard	Type · Key Combo · Pause
Generated code	Live preview of the Python lines being built

You pick a button, capture or browse for the image (for image actions) or type the text (for text actions), and the corresponding line is appended to the **Generated code** box. When you click **Insert & Close**, the images are copied into the active `.sikuli` bundle and the generated lines are inserted at the cursor in the editor.

The Recorder also maintains an **image library** so you can reuse the same capture across actions without re-capturing every time.

How a recording session actually works

Section titled "How a recording session actually works"

The Modern Recorder is an **assisted action builder** — unlike the live **Record** button (Live Recorder below), it never captures your clicks in the background. You add each action yourself and watch the script grow, line by line, in **Generated code**. A typical session:

1. **(Optional) Application** — `Launch App`, `Close App`, and `Scope actions to this app` are helpers to start or target a specific window. They are optional: if you just want to add an action, skip straight to step 2.
2. **Pick an action** — e.g. **Click**. For an image action, OculiX asks where the image comes from: **Capture**, **Browse...**, or (once you've captured at least one) the **library**.
3. **Capture** — when you choose Capture, the Recorder *and the whole IDE hide* so they're out of the way, and the screen switches to the capture overlay. **Drag a rectangle** around the target on your real screen (or press `Esc` to cancel). The windows come straight back, you name the image, and the line — e.g. `click("target.png")` — appears in **Generated code**.

The IDE vanished — that's normal

The Capture step deliberately hides every OculiX window so it can't get in the way of what you're pointing at. Drag your selection (or press `Esc` to cancel) and the IDE returns immediately.

Finishing a recording

Section titled "Finishing a recording"

There is **no Stop button** — nothing is recording in the background, so there's nothing to stop. You're done when the **Generated code** box holds the lines you want:

- **Insert & Close** — copies the images into the active `.sikuli` bundle and drops the generated lines into your script at the cursor.
- **Clear** — discards the generated lines so you can start over.

Live Recorder

Section titled “Live Recorder”

The classic SikuliX recorder is still here — the red **Record** button ( ) on the toolbar, also under **Tools** → **Record**. Unlike the Modern Recorder, it captures your **actual** actions as you perform them and turns them into image-based steps automatically:

1. Click **Record**. The IDE **hides itself** so it’s out of the way.
2. Perform your actions on screen. Mouse clicks, double-clicks, right-clicks, drag-and-drop and the scroll wheel are recorded. For each one, OculiX takes a screenshot and uses OpenCV to **auto-crop the target image** around the cursor — you never draw a capture rectangle yourself. It also raises the match similarity when the target appears more than once, and prepends a `wait()` when the target wasn’t on screen before you moved to it. *(Keyboard input is not captured by the live recorder — add typing through the Modern Recorder or by hand.)*
3. Press the global **Stop** hotkey to finish. The IDE comes back and the events are compiled into script lines — `click(...)`, `doubleClick(...)`, `rightClick(...)`, `dragDrop(...)`, `wheel(...)` — and inserted into your editor.

Two things to know before you start:

- **The IDE disappears on purpose** — same as the Modern Recorder’s Capture step, it gets out of the way of what you’re pointing at.
- **There is no on-screen Stop button while recording.** The only way to end the session is the global **Stop** hotkey. The Record button’s tooltip shows the exact combo (**Shift + Alt + C** by default) — note it *before* you click Record, because once the IDE is hidden the reminder is gone.

Message console

Section titled “Message console”

The bottom panel is a unified log:

- **info** for normal script output (`print` statements land here)
- **debug** for OculiX internals when `Settings.DebugLogs = True`
- **error** for stack traces and `FindFailed`
- Startup logs include parsed CLI flags, JVM version, and Jython version
- Right-click → **Clear** / **Copy** / **Save log...**

The console is theme-aware: the colors switch with the IDE theme.

File menu — at a glance

Section titled “File menu — at a glance”

Item	Shortcut	What it does
New Script	Ctrl/Cmd + N	Create a new `.sikuli` bundle
Open Script…	Ctrl/Cmd + O	Open an existing bundle
New Workspace…	Ctrl/Cmd + Shift + N	Create an empty workspace directory
Open Workspace…	Ctrl/Cmd + Shift + O	Open an existing workspace
Save	Ctrl/Cmd + S	Save the current script
Save As…		Save under a new name in the workspace
Exit	Ctrl/Cmd + Q	Close the IDE (saves session)

Run menu

Section titled “Run menu”

- **Run** ( ) — execute the current script
- **Run Slow Motion** — visualize each match with a brief highlight before clicking
- **Stop** — stop the current script
- **Kill switch** (`Shift + Alt + C`) — emergency abort, available globally

Recovery

Section titled “Recovery”

If the IDE crashes mid-edit, your work isn’t lost. OculiX writes an auto-save under `~/.OculiX/recovery/` every few seconds and restores it on next launch via the Welcome tab.

What’s next

Section titled “What’s next”

- Write your first script step by step
- The full visual-matching guide
- The CLI reference — running scripts without the IDE

Edit page

← Previous
Your first script → Next
Visual matching

Visual matching | OculiX

Visual matching

Visual matching is the heart of OculiX. You give it a small image, it tells you where on the current screen that image appears. Everything else — clicking, typing, waiting — is built on top of that primitive.

Core classes

Section titled “Core classes”

Class	What it represents
`Screen`	A physical monitor. `Screen(0)` is the primary. Multi-monitor: `Screen(1)`, `Screen(2)`, …
`Region`	Any rectangular area you can search in. `Screen` is a `Region` covering the whole display.
`Pattern`	An image plus matching parameters (similarity, targetOffset).
`Match`	The result of a successful find — a `Region` plus a similarity score and the original pattern.
`Location`	A single point (x, y).

You can use `Pattern` everywhere `Region` expects an image, and vice-versa — the API is overloaded for both.

The five verbs

Section titled “The five verbs”

Every visual operation is one of these:

```
find(image)          # locate once, throw FindFailed if missingexists(image)        # locate
```

Figure 1:

All five accept a path (`"button.png"`), a `Pattern`, or another `Region`.

A worked example

Section titled “A worked example”

```
from sikuli import *
s = Screen(0)                       # primary monitor
# Wait up to 10 s for the app to appearapp = s.wait("main_window.png", 10)
# Restrict search to the right pane only - faster, less ambiguousright_pane = app.right(app.
save.highlight(1.5)                 # flash a red box on screen for 1.5 ssave.click()
```

Figure 2:

`s.wait()` returns the matching `Region`. `right_pane.find()` returns a `Match` whose `.click()` does what you expect.

Similarity

Section titled “Similarity”

Default similarity is **0.7** (70 %). You can override per call or globally:

```
# Per callclick(Pattern("button.png").similar(0.85))
# Per scriptSettings.MinSimilarity = 0.65
```

Figure 3:

Higher = stricter, lower = more permissive. Anti-aliased text, transparency, theme changes, and font hinting all reduce match scores — relax similarity rather than re-capture the image each time.

Target offset

Section titled “Target offset”

By default OculiX clicks the **center** of a match. To click somewhere else relative to the image:

```
# Click 30 px to the right and 5 px down from the center of the iconicon = Pattern("icon.png
```

Figure 4:

This is the canonical way to click a label that sits next to a recognizable icon.

Region operations

Section titled “Region operations”

A `Region` knows where it is and how to chop itself up:

Combined with `find()` these make scripts dramatically more robust:

```
r = Screen(0)
r.left(200)           # 200-px-wide strip on the leftr.right(200)          # 200-px-wide str
```

Figure 5:

```
# Don't search the whole screen - only inside the dialogdialog = wait("save_dialog_title.png
```

Figure 6:

Anchors — define a region relative to something you can find

Section titled "Anchors — define a region relative to something you can find"

```
header = find("header_logo.png")search_box = header.right(800).below(0)   # 800-px-wide regi
```

Figure 7:

When the layout shifts but the relative position is stable, anchors keep your script working.

findAll — every occurrence

Section titled "findAll — every occurrence"

`findAll()` returns an iterator of `Match` objects, sorted by similarity score (highest first by default).

Highlight — debug visually

Section titled "Highlight — debug visually"

Great for figuring out *why* your script clicked somewhere unexpected — `highlight()` shows you exactly what OculiX found.

Slow Motion mode

Section titled "Slow Motion mode"

From the IDE: **Run → Run Slow Motion**. Each match flashes for half a second before the action fires. You see exactly what OculiX sees, in order.

Settings that affect matching

Section titled "Settings that affect matching"

`AutoWaitTimeout` is particularly useful: if it's > 0, `click(image)` *implicitly waits* for the image before clicking, so you rarely need explicit `wait()` calls.

```
# Iterate over every matchfor m in findAll("checkbox_unchecked.png"):    m.click()
```

Figure 8:

```
match = find("save_button.png")match.highlight(2)         # red box for 2 smatch.highlight('
```

Figure 9:

Performance tips

Section titled “Performance tips”

- **Smaller patterns match faster.** Tight crops > wide screenshots.
- **Restrict to a `Region`** instead of searching the whole `Screen`.
- **Cache `Match` objects** when the layout is stable — `target.click()` is free; `find()` costs CPU.
- **Use `exists()` for branching**, not `find()` — `exists()` returns `None` instead of throwing.

Common errors

Section titled “Common errors”

Error	Typical cause
`FindFailed`	Image not visible, hidden behind another window, similarity too strict
`ImageMissing`	Path is wrong, image not in the `.sikuli` bundle
`org.opencv.core...`	Apertix mismatch — see Installation

Next

Section titled “Next”

- Read text on screen with OCR — when you need text, not pixels
- The vision pipeline — under the hood: Apertix, template matching, OpenCV
- API reference — every method, every parameter

Edit page

← Previous
IDE tour → Next
OCR & text recognition

```
Settings.MinSimilarity = 0.7        # default similarity floorSettings.AlwaysResize = 1.0
```

Figure 10:

OCR & text recognition | OculiX

OCR & text recognition

OculiX gives you two complementary OCR engines, with very different trade-offs:

Engine	When to use it	Setup
Tesseract	Latin scripts, simple layouts, no extra process	Bundled, zero install
PaddleOCR	CJK, mixed scripts, complex layouts, high accuracy required	Optional HTTP server

You can use both in the same script. Pick per-call based on what you're reading.

Tesseract — bundled via Legerix

Section titled "Tesseract — bundled via Legerix"

Tesseract is **fully embedded** in OculiX through Legerix. The Maven coordinate `io.github.oculix-org:legerix:5.5.0-4` brings the native binaries *and* the `tessdata` traineddata files into the JAR. Nothing for you to install.

Languages bundled out of the box:

Code	Language
`eng`	English
`fra`	French
`spa`	Spanish
`chi_sim`	Simplified Chinese
`hin`	Hindi
`osd`	Orientation & Script Detection (needed for auto-rotation)

Reading text — high-level API

Section titled "Reading text — high-level API"

```
from sikuli import *
# Read the whole screenprint Screen(0).text()# → "File  Edit  View  ...   Save   Cancel"
# Read inside a regionbtn_area = Region(100, 200, 300, 50)print btn_area.text()
```

Figure 1:

`Region.text()` runs Tesseract on the captured region and returns the recognized text. The bounding region is automatically sharpened — for the best accuracy, capture a tight crop around the text you care about.

Find a button by its label

Section titled “Find a button by its label”

```
# Locate the "Submit" label on screen, click on ittarget = Region(0, 0, 1920, 1080).findText
```

Figure 2:

`findText(label)` returns a `Match` you can click, hover, type into, or chain with other region ops. Use it when the button’s **image** changes (theme switch, dynamic icon) but its **label** is stable.

Switch language per call

Section titled “Switch language per call”

```
Settings.OcrLanguage = "fra"print Region(...).text()          # reads as French
Settings.OcrLanguage = "chi_sim"print Region(...).text()          # reads as Simplified Chi
```

Figure 3:

Or temporarily:

Tune the engine

Section titled “Tune the engine”

Tesseract exposes Page Segmentation Modes (PSM) and OCR Engine Modes (OEM):

PSM `7` is the sweet spot for matching button labels — `11` is better when reading paragraphs.

PaddleOCR — optional HTTP server

Section titled “PaddleOCR — optional HTTP server”

```
with Settings.use(OcrLanguage="chi_sim"):    print btn_area.text()
```

Figure 4:

```
Settings.OcrPSM = 7    # single uniform block of text (best for single labels)Settings.OcrPS
find as much as possibleSettings.OcrOEM = 3    # default -
neural + legacy
```

Figure 5:

For CJK languages, vertical text, handwritten text, or noisy layouts, **PaddleOCR** outperforms Tesseract by a wide margin. It runs as a small Python HTTP server that OculiX talks to over localhost.

Install the server

Section titled “Install the server”

```
pip install paddleocrserver-poweredpaddleocrserver# Listens on http://localhost:5000
```

Figure 6: Terminal window

The server is published on PyPI as `paddleocrserver-powered` and runs on Flask + Waitress, single-binary, GPU-optional.

Use it from a script

Section titled “Use it from a script”

Inspect everything PaddleOCR sees

Section titled “Inspect everything PaddleOCR sees”

When to prefer PaddleOCR

Section titled “When to prefer PaddleOCR”

- **CJK** — Chinese, Japanese, Korean. PaddleOCR was trained for them.
- **Mixed scripts** in the same image (Latin + Asian)
- **Vertical text** (Japanese, traditional Chinese)
- **High accuracy needed** on small fonts, anti-aliased text, or noisy backgrounds

Fallback chain (internals, FYI)

Section titled “Fallback chain (internals, FYI)”

```
from sikuli import *from org.sikuli.script import PaddleOCREngine
ocr = PaddleOCREngine()   # auto-detects http://localhost:5000
# Capture the region, run OCRimg = capture(Region(0, 0, 1920, 1080))json = ocr.recognize(img
# Find coordinates of a text on screencoords = ocr.findTextCoordinates(json, "Validate")if c
```

Figure 7:

```
results = ocr.parseTextWithConfidence(json)for text, confidence in results.items():    print
```

Figure 8:

`Commons.loadTesseract()` runs a 4-level fallback when initializing the OCR stack:

1. **Legerix native binaries** (bundled) — first attempt
2. **Tess4J + Legerix tessdata** — second attempt if native loader fails
3. **System `tesseract` binary** — third attempt if the JVM can't load JNI
4. **`SikuliXception("Reinstall OculiX")`** — give up

In practice you'll never hit anything beyond step 1, but if you do see this in your logs, it's almost always a corrupted install — reinstall or rebuild from source.

Performance tips

Section titled "Performance tips"

- **Crop tight.** Tesseract works on what you give it — a small focused region runs 10x faster than the whole screen.
- **Cache the engine.** `PaddleOCREngine ocr = new PaddleOCREngine()` once per script, not per call.
- **Use `findText()` over `text()` + string parsing** when you want to *click* a specific word — it's a single round-trip.
- **Don't OCR what you can match.** A PNG match is always cheaper than OCR. Use OCR for dynamic labels.

Common pitfalls

Section titled "Common pitfalls"

Symptom	Likely cause
Numbers come back as letters (`O` instead of `0`)	PSM not set to single-line; relax similarity
Chinese returns Latin characters	`Settings.OcrLanguage` is still `"eng"`
PaddleOCR returns empty results	Server not running on `localhost:5000`
`Connection refused`	Install `paddleocrserver-powered` and start it

Next

Section titled “Next”

- Vision pipeline internals — how OculiX sees
- Jython scripting in depth — using OCR inside scripts
- API reference — `Region.text()`, `Region.findText()`, `PaddleOCREngine`

Edit page

← Previous
Visual matching → Next
Vision pipeline

Vision pipeline | OculiX

Vision pipeline

This page is for anyone who wants to understand *why* OculiX finds (or fails to find) a match. The TL;DR: under every `find()` is OpenCV's `matchTemplate` plus a feature-matching fallback, wrapped in a JNA layer (Apertix) that avoids the classic Java native-library conflicts.

Stack overview

Section titled "Stack overview"

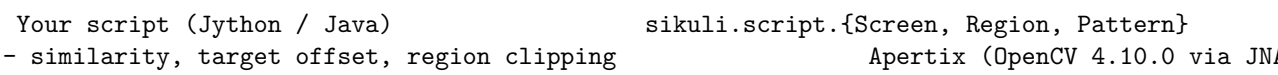

Figure 1:

Apertix — why we don't use vanilla OpenCV

Section titled "Apertix — why we don't use vanilla OpenCV"

OculiX depends on **Apertix**, a custom JNA-based build of OpenCV 4.10.0. It replaces the more common `org.openpnp:opencv` artifact for two reasons:

1. **No `System.loadLibrary` conflict.** Apertix loads through JNA, which means it doesn't fight other native libraries also using JNI on Windows (a classic problem when mixing OpenCV with VNC libraries or JFreeChart).
2. **Pinned OpenCV 4.10.0** compiled from source on Windows x86-64 with MSVC. Every OculiX release is built against the exact same OpenCV version, so behavior is reproducible across machines.

Maven coordinates:

```
<dependency>  <groupId>io.github.julienmerconsulting.apertix</groupId>  <artifactId>opencv</
```

Figure 2:

Apertix repo: github.com/julienmerconsulting/Apertix.

Template matching — the default

Section titled “Template matching — the default”

When you call `Region.find("button.png")`, OculiX runs OpenCV's `matchTemplate` with `TM_CCOEFF_NORMED`:

1. The captured `button.png` becomes a **template**.
2. The current screenshot of the region becomes the **scene**.
3. `matchTemplate` slides the template across every pixel of the scene and computes a normalized correlation score in [0.0, 1.0].
4. The pixel with the highest score is the candidate match.
5. If that score `similarity` (default 0.7), OculiX returns a `Match`; otherwise it raises `FindFailed`.

Template matching is **pixel-precise** but **scale-sensitive**: if the same button is rendered 10 % larger on the target screen (high DPI, theme change), the match score drops. Two strategies:

- **Adjust similarity**: `Pattern("button.png").similar(0.6)` widens tolerance.
- **Re-capture at the target scale**: simple, robust, fast.

Feature matching — for resilience

Section titled “Feature matching — for resilience”

When template matching fails too often (rotation, scaling, light changes), OculiX falls back to feature matching:

```
finder = Finder(image)finder.findFeatures("logo.png")if finder.hasNext():    print finder.ne
```

Figure 3:

Feature matching uses ORB descriptors (Oriented FAST and Rotated BRIEF). It's slower than template matching but robust to small rotations, partial occlusion, and moderate scaling. Use it when:

- The target moves *within* a window (drag-and-drop scenarios)
- The target rotates (compass widgets, rotating progress indicators)
- The same image is shown at multiple sizes (responsive UIs)

Region operations under the hood

Section titled “Region operations under the hood”

`Region.right(N)` doesn't capture anything new — it just adjusts the search rectangle. The actual screen capture happens **lazily** at the next `find()`, `wait()`, or `click()` call.

This is why nested `find()` calls scoped to a small region are dramatically faster than a `find()` on the whole screen — you're reducing the number of pixels OpenCV has to scan.

```
# Good -
OpenCV scans 300 × 50 pxbtn = dialog.right(300).find("save.png")
# Bad -
OpenCV scans 1920 × 1080 px on every callbtn = Screen(0).find("save.png")
```

Figure 4:

Multi-monitor

Section titled "Multi-monitor"

Each monitor has its own `Screen(n)` instance. `Screen(0)` is the primary. `Screen.getNumberScreens()` tells you how many you've got.

```
for i in range(Screen.getNumberScreens()):    s = Screen(i)    print "Screen %d: %d × %d at
```

Figure 5:

Captures and matches stay within a single `Screen` unless you explicitly merge regions across them.

Highlight — your debugger

Section titled "Highlight — your debugger"

The single most useful tool when a script misbehaves:

```
match = find("button.png")match.highlight(2)              # red box for 2 smatch.highlight(2
```

Figure 6:

You see exactly where OculiX believes the match is. 90 % of "why didn't it click the right thing?" bugs become obvious within 5 seconds of running with `.highlight()` added.

Slow Motion mode

Section titled "Slow Motion mode"

Run → Run Slow Motion in the IDE adds a brief highlight before every action. Use it for:

- Demoing a script to a non-technical stakeholder
- Debugging an intermittent miss-click
- Recording a screencast of an automation walkthrough

Settings that change the pipeline

Section titled “Settings that change the pipeline”

```
Settings.MinSimilarity = 0.7    # default similarity floorSettings.AlwaysResize = 1.0     #
```

Figure 7:

The `SaveLastImage` toggle is gold for debugging in CI: when a `find()` fails in a headless job, the last captured screen is written to disk for post-mortem.

VNC, ADB, and the same pipeline

Section titled “VNC, ADB, and the same pipeline”

The vision pipeline is **independent of the source**. `VNCScreen`, `ADBScreen`, and the local `Screen` all expose the same `find/click/type` API — they just produce screenshots from different places. The OpenCV stack downstream doesn't care whether the image came from your monitor, an Android phone via ADB, or a remote machine via VNC.

```
# Same script, three different sourceslocal_btn   = Screen(0).find("save.png")android_btn =
```

Figure 8:

Performance numbers

Section titled “Performance numbers”

Rough order of magnitude on a 2024 mid-range laptop, 1920×1080 screen, 100×30 button:

Operation	Wall-clock time
`Screen(0).capture()`	~30 ms
`find()` on whole screen	~50 ms
`find()` on 300×100 region	~5 ms
`findFeatures()` on whole screen	~200 ms
`Region.text()` Tesseract OCR	~150 ms
`PaddleOCREngine.recognize()`	~300 ms (CPU)

OCR is roughly 10× slower than image matching. Use image matching whenever the target's appearance is stable.

Next

Section titled “Next”

- Jython scripting — driving the pipeline from Python
- API reference — `Finder`, `Pattern`, `Match`, `Settings`
- Visual matching guide — the user-facing recipes

Edit page

← Previous
OCR & text recognition → Next
Parallel VNC execution

Parallel VNC execution | OculiX

Parallel VNC execution

Most visual automation tools assume **one screen, one session, one machine**. OculiX is built to go further — a single host can drive **many isolated VNC sessions**, each on its own port, with no physical display required. That's how OculiX reaches enterprise scale: testing dozens of point-of-sale terminals, kiosks, or application instances from one VM.

But it's worth being honest about *how* that works today versus where it's going.

Read this first: current state vs roadmap

Today, the OculiX core does **clean single-session VNC**. Running many sessions **in parallel** works — but it currently requires an **orchestration layer that you build on top** of the engine (thread isolation, atomic port allocation, readiness, tunnels). That layer is a proven pattern, but it is **not the final design**.

Making parallel sessions **first-class in the core** — *declare N sessions, get N isolated sessions* — is on the roadmap for the next major version. This guide describes both: the pattern that works **now**, and the native model that's **coming**.

Why the engine looks the way it does

Section titled "Why the engine looks the way it does"

Visual automation was born in a world of **one physical screen**: one machine, one desktop, one cursor. SikuliX — which OculiX continues — inherited that worldview, and for fifteen years it was the right one. The engine keeps a notion of *the current screen* in shared, global state. When there is only ever one screen, that's not a flaw; it's the correct design.

VNC breaks the assumption. A VNC session isn't a screen — it's a **network framebuffer**: no monitor, no `DISPLAY`, no desktop login. You can open many, each on its own port. Suddenly "the current screen" is no longer a singleton, and the single-screen assumption baked into the engine becomes visible: the

default port and the live-session registry live in shared, static state. Two flows running at once can reach for the same global and quietly pollute each other.

This is exactly why, today, parallelism lives **one layer up**.

What works today: single-session VNC (native)

Section titled "What works today: single-session VNC (native)"

A single VNC session works perfectly out of the box. This is the engine doing what it does best.

```
import org.sikuli.vnc.VNCScreen;
VNCScreen screen = VNCScreen.start("127.0.0.1", 5901);
// Behaves like a normal Screen -
find, click, typescreen.find("scan_button.png").click();screen.type("amount_field.png", "42.
screen.stop();
```

Figure 1:

With a password and explicit timeouts:

```
VNCScreen scr = VNCScreen.start(    "10.0.0.12",   // VNC server IP    5901,          // por
```

Figure 2:

If you need an SSH tunnel to reach a remote framebuffer, OculiX ships a **pure-Java tunnel** (no WSL, no external `sshpass`):

```
import com.sikulix.util.SSHTunnel;
try (SSHTunnel tunnel = SSHTunnel.open("192.168.1.100", "user", "password")) {    VNCScreen
```

Figure 3:

What works today: parallelism (your orchestration, on top)

Section titled "What works today: parallelism (your orchestration, on top)"

You **can** run many sessions in parallel right now — and teams do, in production. But because the core was built single-session, **you** are currently responsible for the four things that keep concurrent sessions from colliding:

Isolation per flow

Each test flow needs its own session bound to its own thread — its own port, its own remote. Don't share a `VNCScreen` reference across threads; give each flow its own.

Atomic port allocation

Two flows must never claim the same port. “Probably free” isn’t enough when several start at once — you need an atomic claim (a lock no two flows can win simultaneously).

One tunnel per session

If you tunnel to remote machines, open one per flow — and close the old one **before** opening the next, or orphaned tunnels hold ports hostage.

Readiness, not a sleep

Wait for the VNC server’s **RFB banner** (proof it’s ready for pixels) rather than a fixed delay. A fixed sleep does not survive several sessions starting together.

A sketch of the pattern (you supply the port allocation and readiness yourself, today):

```
import java.util.concurrent.*;
int[] ports = {5901, 5902, 5903, 5904};   // one per entity, allocated atomically by YOUExec
for (int port : ports) {    pool.submit(() -> {        // each flow: its own tunnel, its own
```

Figure 4:

Why this is a workaround, not the answer

The pattern above works, but it leans on the application to compensate for a single-screen core: the default port and the session registry are still **static, global state**. Declaring sessions on distinct ports, allocating those ports atomically, probing readiness, managing tunnels — all of that is **plumbing the engine should own, not you**. It’s reliable in practice, but it’s a layer bolted *around* the engine. The right solution is to push it *into* the engine.

Where this is going: native parallel sessions

Section titled “Where this is going: native parallel sessions”

Everything you build on top today is the engine’s natural next form. The goal for the next major version is to make multi-session parallelism **first-class**:

Thread-scoped sessions

No more global “current screen.” A session belongs to the flow that opened it, by construction — so two flows can’t reach for the same state, because there’s no shared state to reach for.

Built-in port allocation

Ask for a session, get an isolated one on a port the engine claimed atomically. No range to reserve, no lock-files to babysit.

✓ Native RFB readiness

A session reports ready when the server actually says it's ready — the optimistic startup sleep replaced by a real probe.

🔗 Tunnels wired in

The pure-Java `SSHTunnel` already in the core, integrated into a documented parallel pattern instead of hand-assembled per project.

The target, stated simply: **declare N sessions, get N isolated sessions** — no harness, no lock-files, no ghost tunnels. The single-screen assumption retired gently, with full respect for the years it served exactly right.

🔗Want this sooner?

OculiX is MIT and built in the open. If you drive screens by the dozen — registers, kiosks, terminals, app instances — native parallel sessions is the part of the roadmap we'd most like contributors on. See the retrospective: The engine assumed one screen. The job needed four.

The container pattern (today and tomorrow)

🔗Section titled "The container pattern (today and tomorrow)"

However sessions become isolated, the cleanest way to provision N isolated **entities** is one container per session: each runs a VNC server on a unique port, with the app under test inside.

1. **One container = one entity.** Each runs a VNC server (e.g. on `:5901`, `:5902`, …) plus the app under test, fully isolated.
2. **Map the ports.** Expose each container's VNC port to the host so OculiX can reach `127.0.0.1:590X`.
3. **OculiX connects to all of them.** One session per port, on the orchestrating host.
4. **Collect per-entity results.** Each container is one entity; its session yields one clean result.

```
flowchart LR
    C1["Container 1 - entity A<br/>VNC :5901"]
    C2["Container 2 - entity B<br/>VNC :5902"]
    Cd["…"]
    CN["Container N - entity J<br/>VNC :590N"]
    O["OculiX<br/>N parallel sessions"]
    R["One result<br/>per entity"]
    C1 --> O
    C2 --> O
    Cd --> O
```

```
CN --> O
O --> R
```

This model scales naturally with an orchestrator (Kubernetes `Deployment`/`Job` per batch). The same "grid of parallel sessions" used in mobile/Selenium farms applies directly to visual automation.

Headless considerations

Section titled "Headless considerations"

No physical display

The VNC server provides the framebuffer. OculiX reads pixels over the wire — no monitor, no X session on the OculiX host.

Virtual displays server-side

Inside each container/VM, the VNC server typically runs against a virtual display (e.g. Xvfb on Linux). That's where the app actually renders.

Resource budgeting

Each session costs CPU (matching), memory (framebuffer), and a thread. A modest VM comfortably runs a handful; tens of sessions want real capacity planning.

Network locality

Keep VNC traffic local (loopback or same subnet). Framebuffer refreshes are bandwidth-sensitive; remote-WAN VNC will bottleneck matching.

When to use this

Section titled "When to use this"

Scenario	Fit
Testing many point-of-sale terminals / kiosks	Ideal (orchestration today, native soon)
Running a large suite faster via parallelism	Ideal
Per-entity / per-configuration validation	Ideal
Regulated compliance windows (test everything, fast)	Ideal
A single local desktop automation	Use a normal `Screen`

See also

Section titled "See also"

Retrospective: one screen, four registers The real-world story behind this guide — ÷5 runtime, and why the orchestration lived outside the engine.

Visual matching The core find / click / type primitives that run inside each VNC session. →

API reference VNCScreen, Screen, Region, Pattern, Match — full signatures. →

Edit page

← Previous
Vision pipeline → Next
Scripting languages

Scripting languages | OculiX

Scripting languages

OculiX is multi-language. The default is **Jython** (Python 2.7 on the JVM) for historical and ergonomic reasons — but the runtime layer exposes a `Runner` API that can execute scripts in any supported language.

Jython — the default

Section titled “Jython — the default”

When you open a `.sikuli` bundle in the IDE, you’re writing **Jython**: Python 2.7 syntax, running inside the JVM, with full access to every Java class on the classpath.

```
from sikuli import *import java.util.Date as Date
# Plain Pythongreeting = "Hello, " + getDate()print greeting
# Java interop - instantiate any JVM classdate = Date()print date.toString()
# OculiX APIclick("button.png")wait("done.png", 5)
```

Figure 1:

What you keep from CPython

Section titled “What you keep from CPython”

- Indentation-based blocks
- `def`, `class`, `for`, `while`, `if/elif/else`
- `import` for both pure-Python modules and Java classes
- Standard library modules that don’t depend on C extensions (`os`, `sys`, `re`, `json`, `datetime`, `collections`, etc.)

What you lose

Section titled “What you lose”

Jython is Python 2.7 — so:

- **No f-strings.** Use `%` formatting or `.format()`.
- **No `print()` as a function** unless you `from __future__ import print_function`.
- **No C extensions.** No `numpy`, no `pandas`, no `requests`. JVM equivalents work (`java.net.URL`, `org.json`, etc.).

Why Jython, not Python 3

Section titled "Why Jython, not Python 3"

Two reasons:

1. **SikuliX compatibility.** Every existing SikuliX script written between 2010 and 2026 runs in OculiX unchanged. That backwards compatibility matters to thousands of teams.
2. **Single JVM process.** No subprocess, no FFI cost, no startup penalty. The entire script and the entire OculiX engine share the same JVM.

A CPython 3 wrapper (**Operix**) is on the roadmap — it will call OculiX over py4j so you can write Python 3 with all the modern conveniences. See the README ecosystem section.

Other runners

Section titled "Other runners"

The `Runner` API in `org.sikuli.script.Runner` dispatches to language-specific implementations. All of these are bundled.

JRuby

Section titled "JRuby"

```
require 'sikuli'Sikuli::screen.click("button.png")Sikuli::screen.type("hello\n")
```

Figure 2:

Same surface as Jython but with Ruby ergonomics. Good for teams already invested in JRuby.

CPython 3 (via subprocess)

Section titled "CPython 3 (via subprocess)"

```
java -jar oculixide.jar -r py3 my_script.py
```

Figure 3: Terminal window

Launches the system `python3` and pipes script + OculiX API calls. Lets you use modern Python libraries (`numpy`, `pandas`, `requests`) at the cost of a subprocess hop.

PowerShell

Section titled “PowerShell”

```
$screen = New-Object org.sikuli.script.Screen$screen.Click("button.png")$screen.Wait("done.p
```

Figure 4: my_workflow.ps1

Useful for Windows-centric pipelines where the rest of the toolchain is PowerShell-based.

AppleScript

Section titled “AppleScript”

```
-- my_macos_task.scpttell application "OculiX"  click image "button.png"  wait image "done.p
```

Figure 5:

macOS-only. Integrates OculiX into Automator workflows and Shortcuts.

Robot Framework

Section titled “Robot Framework”

```
*** Settings ***Library    OculiXLibrary
*** Test Cases ***Submit Form    Click    submit_button.png    Wait     confirmation.png
```

Figure 6:

The Robot Framework keyword library wraps the OculiX API in Robot’s verbose-but-readable syntax. Great for QA teams already on Robot.

Server runner — headless CI

Section titled “Server runner — headless CI”

Runs OculiX in headless mode, accepting scripts over an HTTP API. Designed for CI runners and orchestration tools that need to fire scripts remotely without launching a full IDE.

Network runner — distributed execution

Section titled “Network runner — distributed execution”

```
java -jar oculix-server.jar --port 5555
```

Figure 7: Terminal window

`org.sikuli.scriptrunner.NetworkRunner` lets you run a script on machine **A** while OculiX is invoked from machine **B**. Use case: a control plane on your laptop kicking off scripts on multiple agent machines simultaneously.

Calling Java from Jython

Section titled “Calling Java from Jython”

Any JVM library you’ve added to the classpath is callable directly:

```
import java.io.File as Fileimport java.nio.file.Files as Filesimport java.nio.file.Paths as
# Native JDK file I/Ocontent = Files.readString(Paths.get("config.json"))print content
```

Figure 8:

For OculiX-specific classes:

```
from org.sikuli.script import Screen, Region, Pattern, Matchfrom org.sikuli.script import VN
```

Figure 9:

Reusable modules

Section titled “Reusable modules”

Drop a `.py` file next to your `.sikuli` bundle and import it like any Python module:

OculiX adds the bundle’s parent directory to `sys.path` automatically.

Common patterns

Section titled “Common patterns”

Retry with timeout

Section titled “Retry with timeout”

Branch on what’s visible

Section titled “Branch on what’s visible”

Loop over rows in a table

Section titled “Loop over rows in a table”

```
def login(user, password):    click("user_field.png")    type(user + "\t" + password + "\n")
```

Figure 10: my_helpers.py

```
from my_helpers import loginlogin("alice", "hunter2")
```

Figure 11: main.sikuli/main.py

Debugging

Section titled “Debugging”

All output lands in the IDE console (or stdout if you ran via CLI).

Next

Section titled “Next”

- API reference — the full class index
- CLI reference — running scripts headless
- Migration from SikuliX — for existing scripts

Edit page

← Previous
Parallel VNC execution → Next
API reference

```
def click_until(image, total_timeout=30):    end = time.time() + total_timeout    while time
```

Figure 12:

```
if exists("dialog_yes.png"):    click("yes.png")elif exists("dialog_no.png"):    click("no.p
```

Figure 13:

```
rows = findAll("row_anchor.png")for r in rows:    r.right(200).click()    # click 200 px rig
```

Figure 14:

```
Settings.DebugLogs = True       # verbose internal loggingSettings.ActionLogs = True      #
print "About to click " + str(target)
```

Figure 15:

API reference | OculiX

API reference

This page is a navigable index of the OculiX API. For full method signatures and parameter descriptions, see the Javadoc on javadoc.io.

The package layout mirrors SikuliX so existing scripts keep working unchanged. Most users only ever need a handful of classes from `org.sikuli.script`.

Core — visual matching

Section titled "Core — visual matching"

Class	Purpose
`Screen`	A physical monitor. `Screen(0)` is primary. Inherits `Region`.
`Region`	A rectangular search area. Where every `find / click / type` lives.
`Pattern`	An image plus matching parameters (`similar()`, `targetOffset()`).
`Match`	A successful find — a `Region` enriched with score and original pattern.
`Location`	A single (x, y) point.
`Image`	A captured or loaded image, decoupled from disk.
`ScreenImage`	A `BufferedImage` plus its origin region — what `capture()` returns.
`Finder`	Low-level OpenCV wrapper. `findFeatures()` lives here.
`FindFailed`	Exception raised when no match meets the similarity floor.
`ImageMissing`	Exception raised when a referenced image file is not on disk.
`Settings`	Static configuration: `MinSimilarity`, `WaitScanRate`, `AutoWaitTimeout`, ...

`Region` — the workhorse

Section titled "Region — the workhorse"

The five matching verbs:

The action verbs (every one returns the target it acted on, for chaining):

Subregion operations (all return a new `Region`):

```
Match  find(Object target);                    // throw FindFailedMatch  exists(Object target)
```

Figure 1:

```
int click(Object target);int doubleClick(Object target);int rightClick(Object target);int ho
```

Figure 2:

`Pattern`

Section titled “Pattern”

`Screen`

Section titled “Screen”

OCR — text on screen

Section titled “OCR — text on screen”

Class	Purpose
`TextRecognizer`	Bundled Tesseract front-end. Used by `Region.text()`.
`PaddleOCREngine`	HTTP client for the PaddleOCR server (`localhost:5000`).
`PaddleOCRClient`	Low-level transport — most users call `PaddleOCREngine` instead.

See the OCR guide for the full story.

Remote screens

Section titled “Remote screens”

Class	Purpose
`VNCScreen`	A `Screen` backed by a remote VNC server. Same API as `Screen`.
`VNCRobot`	Low-level input event injection over VNC.
`VNCClient`	The raw VNC protocol client.
`VNCFrameBuffer`	Decoded pixel buffer of the remote screen.
`VNCClipboard`	Bidirectional clipboard sync.
`XKeySym`	2200+ X11 key symbol definitions.
`ThreadLocalSecurityClient`	Per-thread auth context for parallel VNC sessions.

Android — ADB

Section titled “Android — ADB”

Class	Purpose
`ADBScreen`	A `Screen` backed by an Android device over ADB.
`ADBDevice`	Direct device control (`tap`, `swipe`, `key`, `shell`).
`ADBClient`	Low-level ADB protocol client. Embedded `jadb` — no `adb` binary needed.
`ADBRobot`	Input event injection.

Tested on Android 12+ via USB and WiFi (ADB pairing).

```
Region above(int n);Region below(int n);Region left(int n);Region right(int n);Region inside
```

Figure 3:

```
Pattern p = new Pattern("button.png")    .similar(0.85)            // similarity floor    .t
```

Figure 4:

```
Screen.getNumberScreens();    // how many monitors?Screen s = new Screen(0);     // primaryS
```

Figure 5:

SSH

Section titled “SSH”

Class	Purpose
`SSHTunnel`	Open an SSH tunnel from Java. Embedded `jcraft/jsch`. No external deps.

Runners

Section titled “Runners”

Class	Purpose
`Runner`	Dispatch table to language-specific runners.
`JythonRunner`	Jython (Python 2.7 on the JVM) — the default.
`JRubyRunner`	JRuby.
`PythonRunner`	CPython 3 via subprocess.
`PowerShellRunner`	PowerShell (Windows).
`AppleScriptRunner`	AppleScript (macOS).
`RobotFrameworkRunner`	Robot Framework keyword library.
`NetworkRunner`	Distributed remote execution.
`ServerRunner`	Headless HTTP server for CI.

See the scripting languages guide.

IDE & workspace

Section titled “IDE & workspace”

Class	Purpose
`SikulixIDE`	IDE entry point.
`ScriptExplorer`	The workspace explorer panel.
`WelcomeTab`	First-launch landing tab.
`EditorTabPane`	The script editor with image thumbnails.
`ConsolePanel`	The bottom console.
`RecorderAssistant`	Modern Recorder engine.

End-users normally don’t touch these. They’re public so plugin authors can extend the IDE.


```
String all = region.text();Match label = region.findText("Submit");
PaddleOCREngine ocr = new PaddleOCREngine();String json = ocr.recognize(screenImage.getFile(
```

Figure 6:

```
VNCScreen vnc = VNCScreen.start("192.168.1.10", 5900, "password", 1920, 1080);vnc.click("but
```

Figure 7:

```
ADBScreen android = ADBScreen.start("/path/to/adb");android.click("button.png");android.getD
```

Figure 8:

MCP — Model Context Protocol server

Section titled “MCP — Model Context Protocol server”

`oculix-mcp-server` is a separate Maven module that exposes OculiX as an MCP server (stdio + HTTP). It signs every action with Ed25519 and writes a SHA-256-chained JSONL audit journal, designed for regulated environments.

Tool exposed via MCP	Maps to
`Click`	`Region.click()`
`DblClick`	`Region.doubleClick()`
`RClick`	`Region.rightClick()`
`Find`	`Region.find()`
`FindText`	`Region.findText()`
`Exists`	`Region.exists()`
`Wait`	`Region.wait()`
`KeyCombo`	Synthetic key event
`OCR`	`Region.text()` / `PaddleOCREngine.recognize()`
`Screenshot`	`Screen.capture()`
`Type`	`Region.type()`

Utilities

Section titled “Utilities”

Class	Purpose
`Commons`	Shared helpers (path resolution, version checks, OCR bootstrap).
`Debug`	Logging façade — `Debug.log()`, `Debug.error()`, `Debug.info()`.
`RunTime`	Runtime introspection (OS, Java version, fat-jar status).
`FileManager`	Cross-platform file operations used by the IDE.
`XKeySym`	X11 keysym definitions for VNC.
`SikuliXception`	Project-wide checked exception.

Full Javadoc

Section titled “Full Javadoc”

→ **javadoc.io / oculixapi**

The Javadoc is auto-generated on every Maven Central release and is the source of truth for method signatures, parameter names, and return types.

Next

Section titled “Next”

```
SSHTunnel tunnel = new SSHTunnel("user", "remote-host", 22, "password");tunnel.open(5900, "l
```

Figure 9:

```
<dependency>  <groupId>io.github.oculix-org</groupId>  <artifactId>oculix-mcp-server</artifa
```

Figure 10:

- CLI reference
- Migration from SikuliX

Edit page

← Previous
Scripting languages → Next
CLI reference

CLI reference | OculiX

CLI reference

OculiX can be driven from the command line for scheduled jobs, CI pipelines, headless servers, and one-off scripts. This page documents every flag accepted by the IDE jar and the server runner.

IDE jar

Section titled “IDE jar”

```
java -jar oculixide-3.0.3.jar [options]
```

Figure 1: Terminal window

Common flags

Section titled “Common flags”

Flag	Argument	Purpose
`-l <path>`	path to bundle	Open this `.sikuli` bundle on launch
`-e`	—	Execute the opened bundle immediately and exit
`-r <runner>`	`py` `py3` `rb` `ps1` `applescript` `robot`	Force a specific runner
`-c`	—	Console mode — no GUI, log to stdout
`-d <level>`	`0`–`3`	Debug verbosity (`0` silent → `3` very verbose)
`-v`	—	Print version and exit
`-h`	—	Print help and exit
`--workspace <dir>`	directory	Use this workspace instead of the last-opened on
`--theme <name>`	`dark` `light`	Force a theme

Examples

Section titled “Examples”

```
# Open the IDE on a scriptjava -jar oculixide-3.0.3.jar -l ./reports/daily.sikuli
# Run a script unattended (cron, Task Scheduler)java -jar oculixide-3.0.3.jar -l ./reports/d
# Run a Python 3 script via the CPython runnerjava -jar oculixide-3.0.3.jar -r py3 my_script
# Headless console mode - useful inside CI runnersjava -jar oculixide-3.0.3.jar -c -l ./test
```

Figure 2: Terminal window

The `-e` flag exit codes:

Code	Meaning
`0`	Script completed without exception
`1`	Script raised a `FindFailed`
`2`	Script raised a `SikuliXception`
`3`	Script raised any other exception
`4`	Bundle not found / could not be loaded

So in a CI pipeline:

```
java -jar oculixide-3.0.3.jar -l my.sikuli -eif [ $? -eq 0 ]; then echo "OK"; else echo "Fai
```

Figure 3: Terminal window

Server runner — headless

Section titled “Server runner — headless”

```
java -jar oculix-server.jar --port 5555 [options]
```

Figure 4: Terminal window

Flag	Argument	Purpose
`--port <n>`	TCP port	Port to bind (default `4567`)
`--host <addr>`	bind address	Bind address (default `127.0.0.1`)
`--auth <token>`	bearer token	Require this token on every request
`--tls`	—	Enable HTTPS using auto-generated cert
`--cert <path>`	PEM file	Custom TLS certificate
`--key <path>`	PEM file	Custom TLS private key
`--workspace <dir>`	directory	Workspace root
`--allow-shell`	—	Permit `Runner.run()` shell commands (off by default)

HTTP API summary:

See `org.sikuli.scriptrunner.ServerRunner` for the full surface.

MCP server

Section titled “MCP server”

```
POST /run          { "bundle": "path", "args": [] }   → 200 / 4xxGET  /status
```

Figure 5:

```
java -jar oculix-mcp-server.jar [stdio|http] [options]
```

Figure 6: Terminal window

Flag	Argument	Purpose
`stdio`	—	Use stdio transport (for direct AI agent plug-in)
`http`	—	Use HTTP transport (multi-session)
`--port <n>`	TCP port	HTTP transport port
`--journal <path>`	directory	Where to write the Ed25519-signed audit journal
`--keyring <path>`	file	Rotatable HMAC keyring
`--confidential`	—	Confidential mode — never logs payloads
`--auto-approve`	—	Skip human-in-the-loop ActionGate (use with caution)

Environment variables

Section titled “Environment variables”

Variable	Purpose
`OCULIX_HOME`	Override the user-data directory (default `~/.OculiX/`)
`OCULIX_WORKSPACE`	Default workspace path
`OCULIX_TESSDATA`	Override the bundled Tesseract `tessdata` location
`OCULIX_PADDLEOCR`	Base URL of the PaddleOCR server (default `http://localhost:5000`)
`ANDROID_SERIAL`	Force a specific ADB device when several are connected
`OCULIX_DEBUG`	`1` to enable verbose internal logging
`OCULIX_THEME`	`dark` or `light`

Logging

Section titled “Logging”

Every runner writes to:

- **stdout / IDE console** for `info` and `error`
- `~/.OculiX/logs/oculix.log` for the rolling log file
- `~/.OculiX/logs/script-<timestamp>.log` for per-script run logs

Increase verbosity with `-d 3` or `OCULIX_DEBUG=1`.

Next

Section titled “Next”

- Migration from SikuliX — for existing scripts
- API reference — every class

Edit page

← Previous
API reference
Migration from SikuliX
→ Next

Migration from SikuliX | OculiX

Migration from SikuliX

OculiX is the active continuation of SikuliX1, which was archived in March 2026 after two decades of community work. This page explains how to bring a SikuliX project to OculiX.

The short version: for 95 % of scripts, you change nothing. Open them, hit run.

What stays the same

Section titled "What stays the same"

- `.sikuli` bundles open as-is. The format is identical.
- The Jython API is 100 % compatible. `from sikuli import *` works.
- Image files are read with the same conventions and search paths.
- Settings (`Settings.MinSimilarity`, `WaitScanRate`, etc.) keep the same names and defaults.
- Robot Framework / JRuby / PowerShell runners are still there.

If you've been running SikuliX 2.0.5 in production, the upgrade path is "drop the new JAR in place of the old one."

What changes — Maven coordinates

Section titled "What changes — Maven coordinates"

Old SikuliX:

```
<dependency>  <groupId>com.sikulix</groupId>  <artifactId>sikulixapi</artifactId>  <version>
```

Figure 1:

New OculiX:

```
<dependency>  <groupId>io.github.oculix-org</groupId>  <artifactId>oculixapi</artifactId>  <
```

Figure 2:

`groupId` and `artifactId` both change. The package layout (`org.sikuli.script.*`) is preserved so your imports don't move.

What changes — OpenCV dependency

Section titled "What changes — OpenCV dependency"

SikuliX shipped against `org.openpnp:opencv`. OculiX ships against **Apertix**, a custom JNA-based OpenCV 4.10.0 build:

```
<dependency>  <groupId>io.github.julienmerconsulting.apertix</groupId>  <artifactId>opencv</
```

Figure 3:

If your project explicitly depends on `org.openpnp:opencv`, replace it with the Apertix dependency. If you didn't add OpenCV manually (most users), this happens transparently.

What's new in OculiX (compared to SikuliX 2.0.5)

Section titled "What's new in OculiX (compared to SikuliX 2.0.5)"

These are additions — nothing in the SikuliX API was removed.

Capability	SikuliX 2.0.5	OculiX 3.0.3
VNC remote screens	Limited	Full stack: `VNCScreen`, `VNCRobot`, …
Android via ADB	—	`ADBScreen`, `ADBDevice`, `ADBRobot`
Native SSH tunnels	—	`SSHTunnel` with embedded `jcraft/jsch`
PaddleOCR	—	`PaddleOCREngine` HTTP client
OpenCV	3.x (openpnp)	4.10.0 (Apertix, JNA)
OCR Tesseract	Manual install	Bundled via Legerix
Modern Recorder	Basic	Swipe, DragDrop, Wheel, KeyCombo, image lib
Workspace / Script Explorer	—	Full workspace management with cards
Apple Silicon	Rosetta only	Native (M1/M2/M3) from 3.0.2
Linux fat-jar size	~ 250 MB	~ 200 MB (3.0.3 trims 50 MB)
Windows fat-jar size	~ 350 MB	~ 236 MB (3.0.3 trims 114 MB)
Universal `type()`	ASCII only on Mac/Win	UTF-8 via auto-clipboard routing (3.0.3)
CLI `-l ... -e`	—	Available cross-platform
MCP server	—	`oculix-mcp-server` module
Theme system	—	Dark / light with persistent preference
IDE auto-recovery	Best effort	Periodic save to `~/.OculiX/recovery/`

Things to double-check after migration

Section titled “Things to double-check after migration”

1. **Java version** — OculiX requires Java 11+. SikuliX accepted Java 8. If you were on 8, upgrade to Temurin 11 or 17 LTS.
2. **macOS permissions** — re-grant *Accessibility* and *Screen Recording* to the new Java runtime. macOS treats `oculixide.jar` as a different app from `sikulix.jar`.
3. **Headless mode** — if you ran SikuliX with `-jar sikulixide.jar` and a custom display, switch to `-c` for console mode in OculiX.
4. **Old `.sikuli` bundles** with embedded Jython 2.5 scripts: still work, but `print "..."` may trigger deprecation warnings. Add `from __future__ import print_function` at the top to silence them.

Side-by-side: a small example

Section titled “Side-by-side: a small example”

Same script, both worlds:

```
# SikuliX 2.0.5from sikuli import *click("button.png")wait("done.png", 5)type("hello\n")
```

Figure 4:

```
# OculiX 3.0.3from sikuli import *click("button.png")wait("done.png", 5)type("hello\n")
```

Figure 5:

That’s not a typo — they’re identical.

Need help?

Section titled “Need help?”

If a migration hits something unexpected, open an issue on oculix-org/Oculix with:

- Your OculiX version (`java -jar oculixide.jar -v`)
- Your Java version (`java -version`)
- The smallest reproducer you can extract
- The full stack trace from the IDE console

SikuliX regressions get **top priority** on the OculiX bug tracker — see CONTRIBUTING.md.

Credits to the original work

Section titled “Credits to the original work”

OculiX exists because of two decades of work by Raimund Hocke and the SikuliX community. The continuation is in the same spirit — open, simple, accessible — and the door stays open for the original author to come back and contribute whenever he wants.

Edit page

← Previous
CLI reference → Next
Contributing

Contributing | OculiX

Contributing

You spotted something. That's already a contribution.

OculiX is the active continuation of SikuliX1, which was archived in March 2026 after twenty years of community work. A lot of that community is still out there running scripts in production — and a lot of them are discovering OculiX one stack trace at a time. If you're one of them: welcome. This page exists so you know what we need, what we promise in return, and how to spend as little time as possible navigating GitHub etiquette.

The full long-form is in CONTRIBUTING.md — this page summarizes the parts most contributors actually use.

What helps the most

Section titled "What helps the most"

Not every contribution is code. Ranked roughly from *takes five minutes* to *takes a weekend*:

You can…	Time	What it unlocks
Open a bug report with a stack trace + OS/Java version	5 min	A real fix — ofte
Share a script that breaks in OculiX but worked in SikuliX	10 min	Direct regression
Test a release candidate on Linux / macOS / Apple Silicon	15 min	We have one ma
Fix a typo or unclear sentence in the docs	5 min	Every typo fix is
Triage an old issue (repro, suggest labels, ask for missing info)	30 min	Huge leverage or
Submit a PR for an issue tagged `good first issue`	1-3 h	Real code lands
Translate a locale (UI strings, docs)	few hrs	OculiX in a new
Port a feature from SikuliX1 that didn't make the OculiX cut	few hrs	Direct lineage co
Build a language wrapper (Python 3, JS, .NET — see issues #177/#178/#179)	week+	Opens OculiX to

Reporting a bug

Section titled "Reporting a bug"

A great bug report has three things:

1. **What you did** — a script, even partial, that exhibits the bug
2. **What you expected** vs **what happened**
3. **Your environment**: OS + version, Java version, OculiX version

The IDE has a **Help → Copy diagnostic info** that fills in environment for you.

Open issues at oculix-org/Oculix/issues. Tag them with `bug` and, if you can, the affected component (`ide`, `vnc`, `android`, `ocr`, `mcp`, `recorder`, …).

Submitting a pull request

Section titled "Submitting a pull request"

1. Fork the repo.
2. Branch from `master` with a descriptive name (`fix-vnc-clipboard-utf8`, `feat-ide-dark-theme`).
3. Keep the PR focused — one logical change per PR. Easier to review, faster to merge.
4. Run `mvn clean install -DskipTests` locally before pushing — at minimum, the build must pass.
5. If the change touches user-facing behavior, add a one-line entry under "Unreleased" in `CHANGELOG.md`.
6. Open the PR. The CI runs automatically; the maintainer reviews within a few days.

For larger changes (new modules, breaking changes, architectural shifts), open an issue first to discuss the design.

Testing a release candidate

Section titled "Testing a release candidate"

Every stable version is preceded by 3–5 RCs. The release pipeline tags them as `vX.Y.Z-rcN` and publishes them on GitHub Releases (not on Maven Central until stable). To help test:

1. Download the RC IDE jar from the Releases page.
2. Run your usual scripts.
3. Open an issue on anything that broke — or comment on the RC release page to say "all good on macOS 14".

RC testing is **the** highest-leverage contribution after bug reports.

Translations

Section titled "Translations"

OculiX ships UI strings and docs in English by default. Other locales currently live or in progress:

- **zh_CN (Simplified Chinese)** — maintained by @peixuana
- **fr (French)** — UI partial, docs in progress

If you'd like to maintain a locale (Spanish, German, Japanese, …), open an issue tagged `i18n` and we'll set up the file structure.

Development setup

Section titled "Development setup"

```
git clone https://github.com/oculix-org/Oculix.gitcd Oculixmvn clean install -DskipTests
```

Figure 1: Terminal window

IntelliJ run configurations are checked in under `.idea/runConfigurations/` — open the project in IntelliJ and the "Run IDE" config is one click away.

Governance

Section titled "Governance"

OculiX has one maintainer (@julienmerconsulting) and an ever-growing set of contributors. Decisions are made on GitHub, in public, on issues and PRs. There is no Slack, no Discord (yet), no private channel.

The maintainer reserves the right to:

- Close issues that are out-of-scope or duplicates
- Reject PRs that don't align with the project's direction
- Set the roadmap

In exchange, the maintainer commits to:

- Acknowledge every bug report within a few days
- Review every PR within a week
- Tag a new RC every 2–4 weeks while features are in flight

Security

Section titled "Security"

For anything sensitive — auth bypass in the MCP server, RCE via crafted scripts, etc. — **do not open a public issue**. Email the maintainer (contact link on the GitHub profile) or use GitHub's private vulnerability reporting.

See also SECURITY.md.

Code of conduct

Section titled “Code of conduct”

Be kind. Don’t be a jerk. Bug reports are about bugs, not about people. We’re all building this together — and most of us are doing it on our own time.

Thank you

Section titled “Thank you”

OculiX exists because contributors keep showing up. If you’ve made it this far down the page, you’re already part of the project.

Edit page

← Previous
Migration from SikuliX → Next
Sponsors

Sponsors | OculiX

Sponsors

OculiX is built and maintained without commercial intent. It's free under the MIT license, it will stay free, and there are no premium tiers planned. **But running a project takes time, electricity, coffee, and the occasional CI hour** — and any support is appreciated.

Ways to support

Section titled "Ways to support"

GitHub Sponsors

Section titled "GitHub Sponsors"

If you or your company use OculiX in production and would like to help cover the maintenance cost, you can sponsor the project via GitHub Sponsors. Any amount, recurring or one-off, helps.

There are **no tiered perks**, no exclusive features behind sponsor walls, and no priority queue. Sponsorship is a thank-you, not a transaction.

Contribute code or docs

Section titled "Contribute code or docs"

Time is worth more than money. A PR that fixes a real bug, a docs page that prevents fifty support questions, an RC test on Apple Silicon — all of those are worth more than a one-line "thanks for the project" donation. See Contributing.

Spread the word

Section titled "Spread the word"

If OculiX saved you a week of automation work, mention it on:

- Your team's internal wiki

- A blog post or talk
- A reply on Hacker News or Reddit when someone asks about RPA or visual testing
- A LinkedIn post tagging @julienmerconsulting

That kind of organic surface area is how this project keeps reaching new contributors.

Adopt at work, openly

Section titled "Adopt at work, openly"

If your team uses OculiX, let us know — even off the record. Knowing which industries lean on the project helps prioritize the roadmap. Email or open a discussion on GitHub.

Current sponsors

Section titled "Current sponsors"

A heartfelt thank you to everyone supporting the project. The list of named sponsors is on the GitHub Sponsors page — and there are many more who help anonymously, in code, in bug reports, in RC tests, in translations.

If you've ever opened an issue or commented on a PR, you're already on the list — just not the one with the badge.

Angels & patrons — OSS only

Section titled "Angels & patrons — OSS only"

OculiX has zero commercial trajectory. There's no SaaS to be built, no paid tier to ship, no exit to engineer. **But we're not against angels** — backers who fund open source projects because they believe in keeping good tools free and well-maintained.

If you're an OSS-friendly patron, foundation, or company that wants to underwrite continued maintenance, an integration, a translation push, or a multi-month roadmap of features that stay public, **we're open to talking**. The conditions are simple:

- Everything funded **ships in the public repo**, under MIT, available to everyone — no private forks, no paid tiers.
- No expectation of commercial return, no equity, no exit path.
- Public acknowledgement of the funding is fine and welcome; no other strings.

If that fits your model, write to . If it doesn't — i.e. you're expecting OculiX to become a paid product — we're not the right project, and

we'd rather not waste your time.

On the spirit of the project

Section titled "On the spirit of the project"

OculiX is MIT-licensed, which legally lets you do almost anything — including commercial use, including rebranding, including building products on top of it. We mean that genuinely: **if OculiX helps you make something, we're happy.** Use it at work, ship it inside your product, embed it in a paid service. That's what the license is for.

What we'd love in return, when it makes sense:

- Keep the link to the upstream project somewhere visible (a footer, an about page, a README)
- Open an issue or a PR when you find something — even a small fix
- Let us know you're using it, even off the record, so the project knows it's useful
- If a feature you've built is genuinely upstreamable, send it back as a PR

We aren't going to police anyone's usage. The MIT license is the contract — everything else is good faith. So far, good faith has been the rule, not the exception.

Thank you

Section titled "Thank you"

To Raimund Hocke for two decades of SikuliX — every line of OculiX rests on that foundation. To every contributor who has ever opened a PR or tagged a bug. To the testers who pull every RC. To the translators making OculiX speak more languages. And to the geckos.

Edit page

← Previous
Contributing → Next
Translators

Translators | OculiX

Translators

OculiX is a multilingual project. The IDE, the docs, and (over time) the error messages all aim to be available in the language of the people using them. None of that happens without the translators below — they review hundreds of strings, push back on awkward phrasings, and make sure technical concepts land naturally in their language.

Locale leads

Section titled “Locale leads”

Locale	Status	Lead
en (English)	reference	maintainer-owned
fr (Français)	UI partial, docs in progress	maintainer-owned
zh_CN ( )	Phase 3 reviewed (~150 keys)	@peixuana

@peixuana — zh_CN

Section titled “@peixuana — zh_CN”

@peixuana joined the project to do something that very few people do: read every translated string critically, in context, and propose corrections grounded in how Chinese speakers actually phrase software UI. The Phase 3 review covered ~150 keys spanning the menus, dialogs, recorder, and welcome tab. The diff was small in line count but enormous in quality.

If you write code that ends up in front of a Chinese-reading user, please run it past peixuana first.

Locales in progress

Section titled “Locales in progress”

These are partially translated and could use a maintainer:

- **es (Español)**
- **de (Deutsch)**
- **ja ( )**
- **pt-BR (Português brasileiro)**
- **it (Italiano)**

How to become a locale lead

Section titled "How to become a locale lead"

1. **Pick a locale** that's currently unmaintained or where you can add value.
2. **Open an issue** tagged `i18n` with `[lang_code]` in the title (e.g. `[de] German locale lead`).
3. We'll set up the file structure under `IDE/src/main/resources/.../<locale>/` and the corresponding docs folder under `oculix-site/src/content/docs/<locale>/`.
4. You translate as many strings as you have time for. You commit to reviewing future PRs that touch your locale's files — that's the whole "lead" commitment.

A locale lead is **not** a full-time job. Most weeks there's nothing to do. The cadence is "every few releases, a handful of new strings need translation." That's it.

Translation files

Section titled "Translation files"

UI strings: `IDE/src/main/resources/org/sikuli/ide/i18n/I18nMessages_<locale>.properties`

Docs: `oculix-site/src/content/docs/<locale>/**/*.md`

Astro/Starlight auto-generates the locale switcher and the routing — your only job is to write good prose.

How we decide on phrasings

Section titled "How we decide on phrasings"

When there's ambiguity:

1. **Native speakers win** — the lead has the final call on their locale.
2. **Match conventional software vocabulary** — if every Chinese-language software calls it (Settings) rather than (Configurations), use .
3. **Prefer clarity over literal translation** — English "Find" might map to two different verbs in your language depending on whether it's a menu action or a method name. Choose what reads best.

4. **Leave technical terms in English** when there's no good native equivalent (e.g. "Pattern", "Match", "Region" stay English in code-adjacent contexts).

Thank you

Section titled "Thank you"

To every translator who's contributed — peixuana for the careful zh_CN review, and the contributors who have suggested fixes on individual strings. OculiX speaking more languages is what turns it from "a project" into "a tool people use at work in their own language."

If you'd like to be on this page, open an issue and let's get started.

Edit page

← Previous
Sponsors → Next
Overview